CS3591
COMPUTER NETWORKS
SET 4 – 20 COMPLETE PRACTICAL ANSWERS
JAVA PROGRAMMING LANGUAGE

Anna University | Regulations 2021 | Fourth Semester

Practical component	Marks
Aim / Principle / Apparatus / Procedure	20
Tabulation / Circuit / Program / Drawing	30
Calculation & Results	30
Viva-Voce	10
Record	10
Total	100

Important notes

This manual follows all 20 numbered questions in the uploaded SET 4 paper, including subparts. Programs are in Java as requested. The paper asks for C in Question 2 and a C client/Java server in Question 15; these have been adapted to Java-only versions. Use the paper's specified languages if your examiner requires strict compliance.

Simulation questions provide methodology and formulas, not fabricated results. Enter measurements from your own NS/Wireshark run. Run servers before clients. Compile with javac FileName.java and run with java ClassName. Replace localhost with the server IP for two computers.

QUESTION 1: UDP ECHO CLIENT AND SERVER

Aim

To send a message using UDP and verify that the echoed text is identical.

Principle

UDP is connectionless. The server returns the same datagram; the client compares both strings.

Apparatus Required

JDK, two terminals, localhost.

Procedure

• Run server first

• run client

• enter text

• compare sent and received strings.

Program / Circuit / Drawing

```
// EchoServer.java
import java.net.*;
class EchoServer {
 public static void main(String[] a)throws Exception{
  DatagramSocket s=new DatagramSocket(5000); byte[] b=new byte[1024];
  DatagramPacket p=new DatagramPacket(b,b.length); s.receive(p);
  String m=new String(p.getData(),0,p.getLength());
  byte[] r=m.getBytes();
  s.send(new DatagramPacket(r,r.length,p.getAddress(),p.getPort()));
  System.out.println("Echoed: "+m); s.close();
 }
}
// EchoClient.java
import java.net.*; import java.util.*;
class EchoClient {
 public static void main(String[] a)throws Exception{
  Scanner sc=new Scanner(System.in); System.out.print("Message: ");
  String sent=sc.nextLine(); DatagramSocket s=new DatagramSocket();
  byte[] b=sent.getBytes();
  s.send(new DatagramPacket(b,b.length,InetAddress.getByName("localhost"),5000));
  byte[] r=new byte[1024]; DatagramPacket p=new DatagramPacket(r,r.length); s.receive(p);
  String got=new String(p.getData(),0,p.getLength());
  System.out.println("Received: "+got);
  System.out.println(sent.equals(got)?"MATCH":"NOT MATCH"); s.close();
 }
}
```

Expected Output / Observation

Client prints MATCH when both strings are equal.

Calculation

Compare with String.equals(). Record sent text, received text and match status.

Result

Thus, the experiment was completed successfully.

Viva-Voce

UDP; DatagramSocket; DatagramPacket; send(); receive().

QUESTION 2: HELLO SERVER USING UDP (JAVA)

Aim

To send “Hello, world!” from a UDP client to a server.

Principle

UDP sends datagrams without connection establishment.

Apparatus Required

JDK and two terminals.

Procedure

• Start server

• run client

• verify message at server.

Program / Circuit / Drawing

```
// HelloServer.java
import java.net.*;
class HelloServer {
 public static void main(String[] x)throws Exception{
  DatagramSocket s=new DatagramSocket(5001); byte[] b=new byte[100];
  DatagramPacket p=new DatagramPacket(b,b.length); s.receive(p);
  System.out.println(new String(p.getData(),0,p.getLength())); s.close();
 }
}
// HelloClient.java
import java.net.*;
class HelloClient {
 public static void main(String[] x)throws Exception{
  DatagramSocket s=new DatagramSocket(); byte[] b="Hello, world!".getBytes();
  s.send(new DatagramPacket(b,b.length,InetAddress.getByName("localhost"),5001));
  System.out.println("Message sent"); s.close();
 }
}
```

Expected Output / Observation

Server displays Hello, world!

Calculation

Expected message length is 13 characters.

Result

Thus, the experiment was completed successfully.

Viva-Voce

UDP has no handshake; port identifies application; localhost is 127.0.0.1.

QUESTION 3: TCP PERFORMANCE ANALYSIS AND UDP CLIENT-SERVER

Aim

To analyse TCP performance using a network simulator. To implement a simple UDP client-server application.

Principle

TCP provides reliable ordered delivery, flow control and congestion control. UDP datagrams carry request and reply messages.

Apparatus Required

NS-2/NS-3, topology script, trace file. JDK, two terminals.

Procedure

• Create sender-router-receiver topology

• set bandwidth/delay/queue

• create TCP flow

• run

• inspect trace.

• Start server

• run client

• enter message

• observe reply.

Program / Circuit / Drawing

```
# NS-2 outline
set ns [new Simulator]
set tr [open tcp.tr w]
$ns trace-all $tr
set a [$ns node]; set r [$ns node]; set b [$ns node]
$ns duplex-link $a $r 2Mb 10ms DropTail
$ns duplex-link $r $b 1Mb 20ms DropTail
set tcp [new Agent/TCP]; $ns attach-agent $a $tcp
set sink [new Agent/TCPSink]; $ns attach-agent $b $sink
$ns connect $tcp $sink
set ftp [new Application/FTP]; $ftp attach-agent $tcp
$ns at 0.5 "$ftp start"; $ns at 4.5 "$ftp stop"
$ns at 5.0 "finish"
proc finish {} {global ns tr; $ns flush-trace; close $tr; exit 0}
$ns run

// SUB-QUESTION (b)
// UDPServer.java
import java.net.*;
class UDPServer {
 public static void main(String[] z)throws Exception{
  DatagramSocket s=new DatagramSocket(5002); byte[] b=new byte[1024];
  DatagramPacket p=new DatagramPacket(b,b.length); s.receive(p);
  String m=new String(p.getData(),0,p.getLength()); System.out.println(m);
  byte[] r=("Received: "+m).getBytes();
  s.send(new DatagramPacket(r,r.length,p.getAddress(),p.getPort())); s.close();
 }
}
// UDPClient.java
import java.net.*; import java.util.*;
class UDPClient {
 public static void main(String[] z)throws Exception{
  Scanner sc=new Scanner(System.in); DatagramSocket s=new DatagramSocket();
  byte[] b=sc.nextLine().getBytes();
  s.send(new DatagramPacket(b,b.length,InetAddress.getByName("localhost"),5002));
  byte[] r=new byte[1024]; DatagramPacket p=new DatagramPacket(r,r.length); s.receive(p);
  System.out.println(new String(p.getData(),0,p.getLength())); s.close();
 }
}
```

Expected Output / Observation

Trace file records packet events, delays and drops. Server prints message; client prints Received: message.

Calculation

Throughput=received bits/time. Loss%=(sent–received)/sent×100. Mean delay=total delay/received packets. Verify request and reply strings.

Result

TCP throughput and loss are evaluated from actual trace data. Thus UDP client-server communication works.

Viva-Voce

TCP; congestion; throughput; RTT. sendto equivalent is DatagramSocket.send; receive uses receive; UDP does not guarantee delivery.

QUESTION 4: HTTP WEB CLIENT USING TCP

Aim

To download a plain HTTP webpage through TCP and save the body as HTML.

Principle

HTTP uses TCP for HTTP/1.1; HTTPS additionally requires TLS.

Apparatus Required

JDK, internet, browser.

Procedure

• Connect to host port 80

• send GET

• buffer response

• separate headers/body

• save body.

Program / Circuit / Drawing

```
// HttpGet.java (plain HTTP only)
import java.net.*; import java.io.*;
class HttpGet {
 public static void main(String[] a)throws Exception{
  String host="example.com";
  try(Socket s=new Socket(host,80)){
   s.getOutputStream().write(("GET / HTTP/1.1\r\nHost: "+host+
    "\r\nConnection: close\r\n\r\n").getBytes());
   ByteArrayOutputStream buf=new ByteArrayOutputStream();
   byte[] b=new byte[4096]; int n;
   while((n=s.getInputStream().read(b))!=-1)buf.write(b,0,n);
   String res=buf.toString("UTF-8"); int k=res.indexOf("\r\n\r\n");
   if(k<0)throw new IOException("Bad response");
   try(FileWriter f=new FileWriter("downloaded.html")){f.write(res.substring(k+4));}
   System.out.println("Saved downloaded.html");
  }
 }
}
```

Expected Output / Observation

Creates downloaded.html; open it in browser. HTTP redirects/HTTPS may need TLS-capable handling.

Calculation

Record HTTP status and downloaded byte count.

Result

Thus webpage content was saved using TCP.

Viva-Voce

HTTP; GET; port 80; HTTPS requires TLS.

QUESTION 5: NETWORK COMMANDS AND FTP WIRESHARK

Aim

To study network commands in Unix/Windows and inspect FTP control traffic in an authorized lab.

Principle

Commands inspect packets, sockets, interfaces, DNS and routes. Plain FTP may expose USER/PASS.

Apparatus Required

Linux/Windows, Wireshark, own lab FTP server and disposable credentials.

Procedure

• Run commands

• record purpose. Capture own FTP test login

• filter tcp.port==21

• Follow TCP Stream.

Program / Circuit / Drawing

```
Linux:
sudo tcpdump -i any
netstat -an       # or ss -tulnp
ip addr           # ifconfig
nslookup example.com
traceroute example.com

Windows CMD:
netstat -an
ipconfig /all
nslookup example.com
tracert example.com

Wireshark filter: ftp || tcp.port == 21
Use only a disposable lab account. Follow TCP Stream to inspect
whether USER/PASS are visible. Never capture others' credentials.
```

Expected Output / Observation

tcpdump captures packets; netstat lists connections; ipconfig/ifconfig shows interfaces; nslookup queries DNS; traceroute/tracert shows route. Plain FTP may reveal test credentials.

Calculation

Tabulate command, OS and use. Record whether disposable FTP credentials are readable.

Result

Thus commands and FTP traffic were examined.

Viva-Voce

FTP control port 21; secure alternatives SFTP/FTPS; Wireshark is a packet analyzer.

QUESTION 6: DNS USING UDP WITH ERROR HANDLING

Aim

To resolve a user-given domain to an IP using UDP and report unknown domains.

Principle

DNS maps domain names to IP addresses. This program simulates DNS with a local table.

Apparatus Required

JDK, two terminals.

Procedure

• Start server

• query known and unknown domains from client.

Program / Circuit / Drawing

```
// DNSServer.java
import java.net.*; import java.util.*;
class DNSServer {
 public static void main(String[] a)throws Exception{
  Map<String,String> db=new HashMap<>();
  db.put("example.com","93.184.216.34"); db.put("localhost","127.0.0.1");
  DatagramSocket s=new DatagramSocket(5300);
  while(true){
   byte[] b=new byte[512]; DatagramPacket p=new DatagramPacket(b,b.length); s.receive(p);
   String q=new String(p.getData(),0,p.getLength()).trim();
   byte[] r=db.getOrDefault(q,"NOT FOUND").getBytes();
   s.send(new DatagramPacket(r,r.length,p.getAddress(),p.getPort()));
  }
 }
}
// DNSClient.java
import java.net.*; import java.util.*;
class DNSClient {
 public static void main(String[] a)throws Exception{
  Scanner sc=new Scanner(System.in); System.out.print("Domain: "); String q=sc.nextLine();
  DatagramSocket s=new DatagramSocket(); s.setSoTimeout(3000); byte[] b=q.getBytes();
  s.send(new DatagramPacket(b,b.length,InetAddress.getByName("localhost"),5300));
  byte[] r=new byte[512]; DatagramPacket p=new DatagramPacket(r,r.length); s.receive(p);
  String ans=new String(p.getData(),0,p.getLength());
  System.out.println(ans.equals("NOT FOUND")?"Domain not resolved":"IP: "+ans); s.close();
 }
}
```

Expected Output / Observation

Known domain returns mapped IP; unknown domain prints Domain not resolved.

Calculation

Record domain, returned IP/error.

Result

Thus DNS lookup and error handling were simulated.

Viva-Voce

DNS; UDP/TCP port 53; domain resolution.

QUESTION 7: ARP PROTOCOL

Aim

To map an IPv4 address to a MAC address using an ARP-style client/server simulation.

Principle

ARP resolves IPv4 to MAC on a local LAN. This UDP table simulation does not send real Ethernet ARP frames.

Apparatus Required

JDK, two terminals, IP/MAC table.

Procedure

• Start server

• query IP from client

• inspect returned MAC.

Program / Circuit / Drawing

```
// ARPServer.java
import java.net.*; import java.util.*;
class ARPServer {
 public static void main(String[] a)throws Exception{
  Map<String,String> t=new HashMap<>();
  t.put("192.168.1.10","AA:BB:CC:DD:EE:01");
  DatagramSocket s=new DatagramSocket(5301);
  while(true){byte[] b=new byte[100]; DatagramPacket p=new DatagramPacket(b,b.length); s.receive(p);
   String ip=new String(p.getData(),0,p.getLength()).trim();
   byte[] r=t.getOrDefault(ip,"NOT FOUND").getBytes();
   s.send(new DatagramPacket(r,r.length,p.getAddress(),p.getPort()));}
 }
}
// ARPClient.java
import java.net.*; import java.util.*;
class ARPClient {
 public static void main(String[] a)throws Exception{
  Scanner sc=new Scanner(System.in); System.out.print("IP: ");
  byte[] b=sc.nextLine().getBytes(); DatagramSocket s=new DatagramSocket();
  s.send(new DatagramPacket(b,b.length,InetAddress.getByName("localhost"),5301));
  byte[] r=new byte[100]; DatagramPacket p=new DatagramPacket(r,r.length); s.receive(p);
  System.out.println("MAC: "+new String(p.getData(),0,p.getLength())); s.close();
 }
}
```

Expected Output / Observation

Known IP returns mapped MAC; unknown IP returns NOT FOUND.

Calculation

Record queried IP and returned MAC.

Result

Thus ARP mapping was simulated.

Viva-Voce

ARP; IP-to-MAC; ARP cache.

QUESTION 8: CRC ERROR DETECTION

Aim

To implement CRC for detecting transmission errors.

Principle

CRC uses modulo-2 division (XOR). Sender appends remainder; receiver checks remainder.

Apparatus Required

JDK, binary data and generator.

Procedure

• Enter data/generator

• calculate CRC

• send codeword

• verify received codeword.

Program / Circuit / Drawing

```
// CRC.java
import java.util.*;
class CRC {
 static String div(String x,String g){
  char[] a=x.toCharArray();
  for(int i=0;i<=a.length-g.length();i++) if(a[i]=='1')
   for(int j=0;j<g.length();j++) a[i+j]=(a[i+j]==g.charAt(j))?'0':'1';
  return new String(a,a.length-g.length()+1,g.length()-1);
 }
 public static void main(String[] z){
  Scanner s=new Scanner(System.in); System.out.print("Data: "); String d=s.next();
  System.out.print("Generator: "); String g=s.next();
  String rem=div(d+"0".repeat(g.length()-1),g), code=d+rem;
  System.out.println("CRC="+rem+" Codeword="+code);
  System.out.print("Received codeword: "); String r=s.next();
  String check=div(r,g);
  System.out.println(check.chars().allMatch(c->c=='0')?"No error detected":"Error detected");
 }
}
```

Expected Output / Observation

Zero remainder means no error detected; non-zero remainder indicates error.

Calculation

Record data, generator, remainder, codeword and receiver remainder.

Result

Thus CRC error detection was implemented.

Viva-Voce

CRC; XOR; generator polynomial; CRC detects but does not correct.

QUESTION 9: DISTANCE VECTOR ROUTING AND CHAT SERVER

Aim

To implement Distance Vector shortest routing and a socket chat server (two subparts).

Principle

Distance Vector uses iterative Bellman-Ford updates. Chat uses TCP sockets.

Apparatus Required

JDK, cost matrix, two terminals.

Procedure

• Run routing program with cost matrix. For chat, start server then client and exchange messages.

Program / Circuit / Drawing

```
// DistanceVector.java
import java.util.*;
class DistanceVector {
 public static void main(String[] z){
  Scanner s=new Scanner(System.in); int n=s.nextInt(), inf=999;
  int[][] c=new int[n][n],d=new int[n][n];
  for(int i=0;i<n;i++)for(int j=0;j<n;j++)d[i][j]=c[i][j]=s.nextInt();
  boolean change=true;
  while(change){change=false;
   for(int i=0;i<n;i++)for(int v=0;v<n;v++)for(int j=0;j<n;j++)
    if(c[i][v]<inf&&d[v][j]<inf&&c[i][v]+d[v][j]<d[i][j]){
     d[i][j]=c[i][v]+d[v][j];change=true;}
  }
  for(int i=0;i<n;i++)System.out.println(Arrays.toString(d[i]));
 }
}
// Simple TCP chat server
import java.net.*; import java.io.*;
class ChatServer {
 public static void main(String[] z)throws Exception{
  ServerSocket ss=new ServerSocket(6000); Socket s=ss.accept();
  BufferedReader in=new BufferedReader(new InputStreamReader(s.getInputStream()));
  PrintWriter out=new PrintWriter(s.getOutputStream(),true); String m;
  while((m=in.readLine())!=null){System.out.println("Client: "+m); if(m.equals("exit"))break; out.println("Server: "+m);}
  s.close();ss.close();
 }
}
```

Expected Output / Observation

Routing program prints final cost vectors. Chat server displays messages and replies.

Calculation

Example A-B=2, B-C=3, A-C=10 gives A-to-C cost 5 via B.

Result

Thus Distance Vector and TCP chat server were demonstrated.

Viva-Voce

Bellman-Ford; next hop; ServerSocket; convergence.

QUESTION 10: COMPARE TWO ROUTING PROTOCOLS

Aim

To evaluate two routing protocols using a simulator.

Principle

Distance Vector exchanges neighbour vectors; Link State distributes topology and computes shortest paths.

Apparatus Required

NS-2/NS-3, topology, trace files.

Procedure

• Run same topology and traffic with two protocols

• record convergence, throughput, delay, loss and overhead.

Program / Circuit / Drawing

```
Method:
1. Create a fixed topology and traffic pattern.
2. Run protocol A (e.g. Distance Vector); save trace.
3. Run protocol B (e.g. Link State); save trace.
4. Keep bandwidth, delay, packet size and run time identical.
5. Compare:
   Throughput = received bits / duration
   Loss % = (sent-received)/sent * 100
   Mean delay = total delay / received packets
6. Record convergence time and routing overhead.
```

Expected Output / Observation

Prepare a table of protocol, convergence time, throughput, delay, packet loss and control overhead.

Calculation

Use actual simulator outputs; do not invent measurements.

Result

Thus two routing protocols were evaluated fairly.

Viva-Voce

RIP is Distance Vector; OSPF is Link State; convergence is stable routing state.

QUESTION 11: TCP MULTI-CLIENT ECHO SERVER

Aim

To create a TCP echo server that accepts one or more clients.

Principle

TCP is reliable and connection-oriented. A thread per client enables concurrent service.

Apparatus Required

JDK, server and client terminals.

Procedure

• Run server

• connect one or more clients

• send text

• check echoed text.

Program / Circuit / Drawing

```
// EchoServerTCP.java
import java.net.*; import java.io.*;
class EchoServerTCP {
 public static void main(String[] z)throws Exception{
  ServerSocket ss=new ServerSocket(6001);
  while(true){Socket c=ss.accept(); new Thread(()->{
   try{
    BufferedReader in=new BufferedReader(new InputStreamReader(c.getInputStream()));
    PrintWriter out=new PrintWriter(c.getOutputStream(),true); String m;
    while((m=in.readLine())!=null){out.println(m);if(m.equals("exit"))break;}
    c.close();
   }catch(Exception e){e.printStackTrace();}
  }).start();}
 }
}
// EchoClientTCP.java
import java.net.*; import java.io.*;
class EchoClientTCP {
 public static void main(String[] z)throws Exception{
  Socket s=new Socket("localhost",6001);
  BufferedReader k=new BufferedReader(new InputStreamReader(System.in));
  BufferedReader in=new BufferedReader(new InputStreamReader(s.getInputStream()));
  PrintWriter out=new PrintWriter(s.getOutputStream(),true); String m;
  while(!(m=k.readLine()).equals("exit")){out.println(m);System.out.println("Echo: "+in.readLine());}
  out.println("exit");s.close();
 }
}
```

Expected Output / Observation

Each client receives its own message echoed. Server continues accepting new clients.

Calculation

Count clients/messages and verify sent text equals echo.

Result

Thus multi-client TCP echo service was implemented.

Viva-Voce

Thread; accept(); TCP; echo server.

QUESTION 12: TCP CHAT APPLICATION

Aim

To implement client-server chat using TCP.

Principle

TCP provides reliable, ordered, full-duplex byte-stream communication.

Apparatus Required

JDK, two terminals.

Procedure

• Start server

• run client

• exchange messages

• type exit to end.

Program / Circuit / Drawing

```
// ChatServerTCP.java
import java.net.*; import java.io.*;
class ChatServerTCP {
 public static void main(String[] z)throws Exception{
  ServerSocket ss=new ServerSocket(6002); Socket s=ss.accept();
  BufferedReader in=new BufferedReader(new InputStreamReader(s.getInputStream()));
  PrintWriter out=new PrintWriter(s.getOutputStream(),true);
  BufferedReader kb=new BufferedReader(new InputStreamReader(System.in)); String m;
  while((m=in.readLine())!=null){System.out.println("Client: "+m);if(m.equals("exit"))break;
   System.out.print("Reply: ");out.println(kb.readLine());}
  s.close();ss.close();
 }
}
// ChatClientTCP.java
import java.net.*; import java.io.*;
class ChatClientTCP {
 public static void main(String[] z)throws Exception{
  Socket s=new Socket("localhost",6002);
  BufferedReader kb=new BufferedReader(new InputStreamReader(System.in));
  BufferedReader in=new BufferedReader(new InputStreamReader(s.getInputStream()));
  PrintWriter out=new PrintWriter(s.getOutputStream(),true); String m;
  while(true){System.out.print("You: ");m=kb.readLine();out.println(m);if(m.equals("exit"))break;System.out.println(in.readLine());}
  s.close();
 }
}
```

Expected Output / Observation

Messages appear at both endpoints in sequence.

Calculation

No numerical calculation; verify both sides display the same exchanged messages.

Result

Thus TCP chat was implemented.

Viva-Voce

Full duplex; Socket; ServerSocket; reliable delivery.

QUESTION 13: TCP FILE TRANSFER

Aim

To transfer a user-requested file from server to client using TCP.

Principle

Client requests filename; server validates and sends file bytes. TCP preserves byte order.

Apparatus Required

JDK, shared folder and two terminals.

Procedure

• Place file in shared folder

• start server

• client requests filename

• verify download.

Program / Circuit / Drawing

```
// FileServer.java
import java.net.*; import java.io.*;
class FileServer {
 public static void main(String[] z)throws Exception{
  ServerSocket ss=new ServerSocket(6003); Socket s=ss.accept();
  DataInputStream in=new DataInputStream(s.getInputStream());
  DataOutputStream out=new DataOutputStream(s.getOutputStream());
  String name=in.readUTF(); File f=new File("shared",name);
  boolean ok=f.isFile()&&!name.contains("..")&&!name.contains("/")&&!name.contains("\\");
  out.writeBoolean(ok);
  if(ok){out.writeLong(f.length());FileInputStream fi=new FileInputStream(f);
   byte[] b=new byte[4096];int n;while((n=fi.read(b))!=-1)out.write(b,0,n);fi.close();}
  s.close();ss.close();
 }
}
// FileClient.java
import java.net.*; import java.io.*;
class FileClient {
 public static void main(String[] z)throws Exception{
  Socket s=new Socket("localhost",6003);
  DataOutputStream out=new DataOutputStream(s.getOutputStream());
  DataInputStream in=new DataInputStream(s.getInputStream());
  System.out.print("Filename: ");String name=new BufferedReader(new InputStreamReader(System.in)).readLine();
  out.writeUTF(name);
  if(!in.readBoolean())System.out.println("File not found");
  else{long left=in.readLong();FileOutputStream f=new FileOutputStream("downloaded_"+name);
   byte[] b=new byte[4096];while(left>0){int n=in.read(b,0,(int)Math.min(b.length,left));if(n<0)throw new EOFException();f.write(b,0,n);left-=n;
   f.close();System.out.println("Download complete");}
  s.close();
 }
}
```

Expected Output / Observation

Client creates downloaded_filename or reports File not found.

Calculation

Compare original/downloaded sizes or SHA-256 hashes.

Result

Thus file transfer was completed over TCP.

Viva-Voce

TCP; file length; EOF; filename validation.

QUESTION 14: DNS SIMULATION USING UDP (SECOND VERSION)

Aim

To resolve domains through UDP and report unresolved names.

Principle

A local DNS table maps names to IP addresses; UDP carries request and response.

Apparatus Required

JDK and two terminals.

Procedure

• Run DNS server

• query known and unknown domains.

Program / Circuit / Drawing

```
// DNS server/client structure (Java)
Map<String,String> records = new HashMap<>();
records.put("example.com","93.184.216.34");
records.put("localhost","127.0.0.1");
// Server: DatagramSocket(5304), receive domain,
// lookup records.getOrDefault(domain,"NOT FOUND"),
// send reply to packet.getAddress(), packet.getPort().
// Client: send entered domain to localhost:5304,
// receive reply and print IP or "Domain not resolved".
```

Expected Output / Observation

Known domain returns its configured IP; unknown domain gives error.

Calculation

Record domain and response. This is a simplified DNS simulation.

Result

Thus DNS simulation and error reporting were demonstrated.

Viva-Voce

DNS; UDP; domain-to-IP mapping.

QUESTION 15: JAVA CLIENT-SERVER PROGRAM AND TCP/UDP PERFORMANCE

Aim

To implement a client-server socket application. Both sides are in Java as requested, although paper specifies C client and Java server. To compare TCP and UDP performance using simulation.

Principle

TCP client uses Socket; server uses ServerSocket; both agree on port and message format. TCP retransmits lost data and controls congestion; UDP has no built-in retransmission. Compare throughput, delay, jitter and loss.

Apparatus Required

JDK and two terminals. NS-2/NS-3 and trace files.

Procedure

• Run Java server

• run Java client

• send message

• receive acknowledgement.

• Use identical topology and offered load

• run TCP and UDP flows separately

• extract metrics.

Program / Circuit / Drawing

```
// JavaServer.java
import java.net.*; import java.io.*;
class JavaServer {
 public static void main(String[] z)throws Exception{
  ServerSocket ss=new ServerSocket(6004);Socket s=ss.accept();
  BufferedReader in=new BufferedReader(new InputStreamReader(s.getInputStream()));
  PrintWriter out=new PrintWriter(s.getOutputStream(),true);
  System.out.println("Client: "+in.readLine());out.println("Message received");
  s.close();ss.close();
 }
}
// JavaClient.java
import java.net.*; import java.io.*;
class JavaClient {
 public static void main(String[] z)throws Exception{
  Socket s=new Socket("localhost",6004);
  BufferedReader k=new BufferedReader(new InputStreamReader(System.in));
  PrintWriter out=new PrintWriter(s.getOutputStream(),true);
  BufferedReader in=new BufferedReader(new InputStreamReader(s.getInputStream()));
  System.out.print("Message: ");out.println(k.readLine());
  System.out.println(in.readLine());s.close();
 }
}

// SUB-QUESTION (b)
Metrics:
Throughput (bps) = useful received bits / simulation duration
Packet loss (%) = (sent - received) / sent * 100
Average delay = sum of end-to-end delays / received packets
Jitter = variation in successive packet delays
Keep topology, bandwidth, packet size and duration identical.
```

Expected Output / Observation

Server displays message; client displays Message received. TCP generally provides reliability; UDP may have lower transport overhead but can lose datagrams. Results depend on simulation settings.

Calculation

Verify message and acknowledgement. Fill results using actual trace measurements; do not assume fixed values.

Result

Thus client-server communication was implemented in Java. Paper's C client is adapted per request. Thus TCP and UDP performance were compared.

Viva-Voce

Socket; ServerSocket; TCP port; client/server. TCP reliable; UDP connectionless; jitter; packet loss.

QUESTION 16: CONGESTION CONTROL USING NS

Aim

To simulate a congestion-control mechanism using NS.

Principle

TCP slow start grows congestion window rapidly at first; congestion avoidance grows more gradually. Bottleneck queues may overflow.

Apparatus Required

NS-2/NS-3, Tcl script and trace file.

Procedure

• Create sender-router-receiver

• set bottleneck and queue limit

• attach TCP/FTP

• run and inspect trace.

Program / Circuit / Drawing

```
# NS-2 TCP congestion example
set ns [new Simulator]
set tr [open cc.tr w]; $ns trace-all $tr
set a [$ns node];set r [$ns node];set b [$ns node]
$ns duplex-link $a $r 2Mb 10ms DropTail
$ns duplex-link $r $b 0.5Mb 20ms DropTail
$ns queue-limit $r $b 8
set tcp [new Agent/TCP];$ns attach-agent $a $tcp
set sink [new Agent/TCPSink];$ns attach-agent $b $sink
$ns connect $tcp $sink
set ftp [new Application/FTP];$ftp attach-agent $tcp
$ns at 0.2 "$ftp start";$ns at 4.5 "$ftp stop"
$ns at 5.0 "finish"
proc finish {} {global ns tr;$ns flush-trace;close $tr;exit 0}
$ns run
```

Expected Output / Observation

Trace shows packet events, drops and queueing. Inspect congestion window if configured.

Calculation

Throughput=received bits/time; loss%=dropped/sent×100. Use measured values.

Result

Thus congestion control was simulated using NS.

Viva-Voce

Congestion; slow start; congestion window; bottleneck.

QUESTION 17: DISTANCE VECTOR SHORTEST-PATH SIMULATION

Aim

To find shortest routes using Distance Vector algorithm.

Principle

Routers iteratively update costs using neighbour vectors and Bellman-Ford equation.

Apparatus Required

JDK and cost matrix.

Procedure

• Enter nodes and cost matrix

• use 999 for no link

• repeat updates until stable.

Program / Circuit / Drawing

```
// DistanceVector.java
import java.util.*;
class DistanceVector {
 public static void main(String[] z){
  Scanner s=new Scanner(System.in);int n=s.nextInt(),INF=999;
  int[][] c=new int[n][n],d=new int[n][n];
  for(int i=0;i<n;i++)for(int j=0;j<n;j++)d[i][j]=c[i][j]=s.nextInt();
  boolean change=true;
  while(change){change=false;
   for(int i=0;i<n;i++)for(int v=0;v<n;v++)for(int j=0;j<n;j++)
    if(c[i][v]<INF&&d[v][j]<INF&&c[i][v]+d[v][j]<d[i][j]){
     d[i][j]=c[i][v]+d[v][j];change=true;}
  }
  for(int i=0;i<n;i++)System.out.println(Arrays.toString(d[i]));
 }
}
```

Expected Output / Observation

Final matrix gives minimum costs from every router to every destination.

Calculation

Example A-B=4, B-C=2, A-C=9 gives A-C cost 6 via B.

Result

Thus shortest-path costs were found using Distance Vector.

Viva-Voce

Bellman-Ford; distance vector; route convergence; next hop.

QUESTION 18: ARP CLIENT-SERVER SIMULATION

Aim

To simulate ARP request/reply between client and server.

Principle

ARP resolves local IPv4 to MAC. This is a UDP table simulation, not real Ethernet ARP.

Apparatus Required

JDK and two terminals.

Procedure

• Run server

• enter IP at client

• receive MAC or NOT FOUND.

Program / Circuit / Drawing

```
// Java ARP implementation pattern
Map<String,String> map = new HashMap<>();
map.put("10.0.0.1","00:AA:00:AA:00:01");
// Server binds DatagramSocket(5305), receives IP,
// looks up map.getOrDefault(ip,"NOT FOUND"),
// sends result to requester's address and port.
// Client sends entered IP to localhost:5305 and prints reply.
```

Expected Output / Observation

Known IP returns mapped MAC; unknown IP returns NOT FOUND.

Calculation

Record IP and MAC pair from the lookup table.

Result

Thus ARP request/reply was simulated.

Viva-Voce

ARP maps IP to MAC; MAC is usually 48 bits; ARP cache stores mappings.

QUESTION 19: ERROR DETECTION AND CORRECTION

Aim

To implement an error detection/correction method and identify a one-bit error.

Principle

Hamming(7,4) inserts parity at positions 1,2,4. Syndrome identifies a single-bit error position.

Apparatus Required

JDK and four-bit data.

Procedure

• Enter 4 data bits

• generate parity

• enter received 7-bit word

• calculate syndrome and correct.

Program / Circuit / Drawing

```
// Hamming(7,4) core logic
int[] b=new int[8]; // positions 1..7
// Put data bits at positions 3,5,6,7
b[1]=b[3]^b[5]^b[7];
b[2]=b[3]^b[6]^b[7];
b[4]=b[5]^b[6]^b[7];
// At receiver:
int s1=b[1]^b[3]^b[5]^b[7];
int s2=b[2]^b[3]^b[6]^b[7];
int s4=b[4]^b[5]^b[6]^b[7];
int errorPosition=s1+2*s2+4*s4;
if(errorPosition!=0)b[errorPosition]^=1;
```

Expected Output / Observation

Syndrome zero means no error; non-zero syndrome identifies the bit to flip for a single-bit error.

Calculation

Error position=s1+2s2+4s4. Hamming(7,4) corrects one-bit errors.

Result

Thus one-bit error detection/correction was demonstrated.

Viva-Voce

Parity; syndrome; Hamming(7,4); single-error correction.

QUESTION 20: TCP FILE TRANSFER AND UDP PERFORMANCE

Aim

To transfer a file using TCP and analyse UDP performance using simulation.

Principle

TCP provides reliable file-byte delivery. UDP performance can be measured by throughput, loss and delay.

Apparatus Required

JDK, shared file, NS-2/NS-3, trace.

Procedure

• Run TCP file server/client

• verify file. Then simulate UDP flow and extract metrics.

Program / Circuit / Drawing

```
// File transfer: use FileServer.java and FileClient.java
// from Question 13.
// UDP performance calculations:
double lossPercent = (sentPackets-receivedPackets)*100.0/sentPackets;
double throughputMbps = receivedBits/durationSeconds/1_000_000.0;
double averageDelay = totalDelaySeconds/receivedPackets;
// Use actual counts/times from simulator trace.
```

Expected Output / Observation

Client receives requested file. UDP trace provides sent/received packets and timing.

Calculation

Loss%=(sent−received)/sent×100; throughput=received bits/duration; mean delay=total delay/received packets.

Result

Thus TCP file transfer and UDP performance analysis were completed.

Viva-Voce

TCP reliable; UDP no automatic retransmission; throughput; packet loss.